

Kubernetes Command Cheat Sheet

Every command used across Labs 1–4, in execution order

How to use this sheet

Replace placeholders in angle brackets (e.g. <pod-name>, <revision-number>) with your actual values. Windows cmd.exe-specific syntax is noted where it differs from bash/PowerShell. Run `kubectl get pods` first whenever a command needs an exact pod name.

Cluster Setup

Command	Purpose
<code>kubectl version --client</code>	Confirm kubectl is installed and check its version
<code>kubectl get nodes</code>	Confirm the cluster is up; node should show STATUS Ready

Lab 1 — Foundations: Deployments, Services, Self-Healing

Basic inspection

Command	Purpose
<code>kubectl get pods</code>	List all pods in the current (default) namespace
<code>kubectl get pods --watch</code>	List pods and keep streaming live status changes
<code>kubectl get deployments</code>	List Deployments and their replica status
<code>kubectl get services</code>	List Services and their ClusterIP/type/ports
<code>kubectl get replicaset</code>	List ReplicaSets — the layer between Deployments and Pods

Applying manifests

Command	Purpose
<code>kubectl apply -f <file>.yaml</code>	Create or update objects defined in a YAML file

Self-healing demonstration

Command	Purpose
<code>kubectl delete pod <pod-name></code>	Delete a specific pod; Deployment automatically replaces it

<code>kubectl describe pod <pod-name></code>	Full detail on a pod, including the Events section — the first place to look when debugging
--	---

Imperative scaling

Command	Purpose
<code>kubectl scale deployment <name> --replicas=<n></code>	Change replica count directly, without editing YAML
<code>kubectl describe deployment <name></code>	Shows desired vs. current replica count and recent scaling Events

Lab 2 — ConfigMaps, Secrets, API, Service Discovery

.NET project and image build (run on host machine)

Command	Purpose
<code>docker build -t myapi:latest .</code>	Build the API image from the Dockerfile in the current folder
<code>docker images findstr myapi</code>	Confirm the image exists locally (Windows cmd.exe)
<code>docker inspect myapi:latest</code>	Verify Entrypoint is ["dotnet", "MyApi.dll"], not a debug image
<code>docker run -p 8081:8080 -e ASPNETCORE_URLS=http://+:8080 -e DB_HOST=postgres ... myapi:latest</code>	Local smoke-test only — /db-check will fail here, that's expected (no cluster DNS outside K8s)

Loading the image into a kind cluster (skip if using Kubeadm)

Command	Purpose
<code>docker ps</code>	Find your kind node's container name (ends in -control-plane)
<code>docker save myapi:latest docker exec -i desktop-control-plane ctr -n=k8s.io images import -</code>	Load a host-built image into the kind node's separate image store

Deploying config, secrets, and the app

Command	Purpose
<code>kubectl apply -f 02-configmap.yaml</code>	Create the ConfigMap (non-sensitive config)
<code>kubectl apply -f 02-secret.yaml</code>	Create the Secret (sensitive config, e.g. DB password)
<code>kubectl apply -f 02-postgres.yaml</code>	Deploy Postgres, referencing the Secret
<code>kubectl apply -f 02-api.yaml</code>	Deploy the API, referencing both ConfigMap and Secret
<code>kubectl get configmaps</code>	List ConfigMaps
<code>kubectl get secrets</code>	List Secrets (values are not shown in plain text by default)
<code>kubectl get configmap app-config -o yaml</code>	View the actual key/value contents of a ConfigMap

Proving service discovery

Command	Purpose
---------	---------

<code>kubectl exec -it <pod-name> -- /bin/sh</code>	Open a shell inside a pod (use /bin/sh — slim images often lack bash)
<code>getent hosts postgres</code>	Run inside a pod: confirms DNS resolves the Service name to an IP
<code>echo \$DB_HOST</code>	Run inside a pod: confirms the actual env var value the app sees
<code>kubectl port-forward service/api 8080:8080</code>	Forward a local port to a Service inside the cluster, for browser testing
<code>kubectl logs <pod-name></code>	View a pod's stdout/stderr logs
<code>kubectl logs <pod-name> --tail=20</code>	View only the most recent log lines
<code>kubectl get deployment api -o jsonpath="{.spec.template.spec.containers[0].image}"</code>	Print exactly which image a Deployment is currently configured to run

Lab 3 — Frontend, Ingress, Persistent Storage

Angular build (run on host machine)

Command	Purpose
<code>ng new my-angular-app</code>	Scaffold a new Angular project
<code>ng build</code>	Build for production; outputs static files to <code>dist/</code>
<code>dir dist\my-angular-app</code>	Check the actual build output path (varies by Angular version)
<code>docker build -t myangularapp:latest .</code>	Build the nginx-based frontend image
<code>docker run -p 8082:80 myangularapp:latest</code>	Local smoke-test; /api calls will fail here, that's expected
<code>docker save myangularapp:latest docker exec -i desktop-control-plane ctr -n=k8s.io images import -</code>	Load the frontend image into the kind node (same as the API earlier)

Deploying the frontend

Command	Purpose
<code>kubectl apply -f 03-frontend.yaml</code>	Deploy the Angular frontend Deployment and Service

Installing the Ingress Controller

Command	Purpose
<code>kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.11.2/deploy/static/provider/cloud/deploy.yaml</code>	Install the nginx Ingress Controller — required before any Ingress resource does anything
<code>kubectl get pods -n ingress-nginx</code>	Confirm the controller pod reaches 1/1 Running

Configuring routing

Command	Purpose
<code>kubectl apply -f 03-frontend-ingress.yaml</code>	Apply the frontend routing rule (no rewrite annotation)
<code>kubectl apply -f 03-api-ingress.yaml</code>	Apply the API routing rule (with <code>rewrite-target</code> , kept in its own resource)
<code>kubectl get ingress</code>	List Ingress resources and confirm ADDRESS shows localhost
<code>kubectl describe ingress <name></code>	Detailed routing info, useful when a path isn't matching as expected

Lesson learned the hard way

Putting both the frontend and API rules in one Ingress with a shared rewrite-target annotation breaks the frontend's JS/CSS loading, since annotations apply to the whole Ingress resource, not per path. Always split rules needing different annotation behavior into separate Ingress resources.

Persistent storage for Postgres

Command	Purpose
<code>kubectl apply -f 03-postgres-pvc.yaml</code>	Create the PersistentVolumeClaim
<code>kubectl apply -f 03-postgres-with-storage.yaml</code>	Re-deploy Postgres with the volume mounted
<code>kubectl get pvc</code>	Confirm the claim shows STATUS Bound
<code>kubectl exec -it <postgres-pod> -- psql -U appuser -d appdb</code>	Connect to Postgres directly to test data persistence

Lab 4 — Autoscaling & Rolling Updates

metrics-server (required for HPA)

Command	Purpose
<code>kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml</code>	Install metrics-server
<code>kubectl get pods -n kube-system</code>	Check metrics-server's READY status
<code>kubectl logs -n kube-system deployment/metrics-server</code>	Check for the local-cluster TLS error (cannot validate certificate)

Patching metrics-server for local/kind clusters

Command	Purpose
<code>kubectl patch deployment metrics-server -n kube-system --type=json -p='[{"op": "add", "path": "/spec/template/spec/containers/0/args/-", "value": "--kubelet-insecure-tls"}]'</code>	Windows cmd.exe syntax — note escaped double quotes, not single quotes
<code>kubectl edit deployment metrics-server -n kube-system</code>	Alternative if patch quoting fails — manually add --kubelet-insecure-tls under args:
<code>kubectl top pods</code>	Confirm metrics-server is serving real CPU/memory numbers
<code>kubectl top nodes</code>	Same check, at the node level

HorizontalPodAutoscaler

Command	Purpose
<code>kubectl apply -f 04-api-hpa.yaml</code>	Create the HPA targeting the api Deployment
<code>kubectl get hpa</code>	Quick status: current vs target utilization, current replica count
<code>kubectl get hpa --watch</code>	Live-stream HPA status while load is applied or removed
<code>kubectl describe hpa <name></code>	Full detail: Conditions (why it is/isn't scaling) and Events (scaling history)

Generating and removing load

Command	Purpose
<code>kubectl run load-generator --image=busybox --restart=Never -- /bin/sh -c "while true; do wget -q -O- http://api:8080/ ; done"</code>	Create a disposable pod that continuously hits the API
<code>kubectl delete pod load-generator</code>	Stop the load generator

Rolling updates and rollback

Command	Purpose
<code>docker build -t myapi:v2 .</code>	Build a new version under a distinct tag (never reuse :latest for this)
<code>docker save myapi:v2 docker exec -i desktop-control-plane ctr -n=k8s.io images import -</code>	Load the new version into the kind node
<code>kubectl set image deployment/api api=myapi:v2</code>	Trigger a rolling update to the new image
<code>kubectl rollout status deployment/api</code>	Watch the rollout progress to completion
<code>kubectl rollout history deployment/api</code>	List all revisions (CHANGE-CAUSE is blank unless explicitly recorded)
<code>kubectl rollout history deployment/api --revision=<n></code>	View the full pod template for a specific revision, including its image
<code>kubectl rollout undo deployment/api</code>	Roll back to the previous revision — no YAML editing needed

General Debugging Toolkit (used throughout)

Command	Purpose
<code>kubectl get all</code>	List most object types in the current namespace at once
<code>kubectl delete deployment --all</code>	Remove all Deployments in the current namespace (used to reset between labs)
<code>kubectl delete pod <name></code>	Delete a single pod by name
<code>docker ps</code>	List currently running containers on the host
<code>docker stop <container-id></code>	Stop a container by ID or name
<code>docker rm <container-id></code>	Remove a stopped container

Windows cmd.exe-specific notes

Use `findstr` instead of `grep` for filtering command output.

Multi-line commands using a trailing backslash (`\`) for continuation only work in `bash` — in `cmd.exe`, write the full command on one line instead.

PowerShell uses a backtick (```) for line continuation, not a backslash.

When a JSON patch needs quoting in cmd.exe, use escaped double quotes (\") as the inner quote character, with double quotes as the outer delimiter — not single quotes.